

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**  
**(JUN 2019 - NOV 2019)**

**CS T55 SOFTWARE ENGINEERING**

**V SEMESTER**

## UNIT – I

**Introduction to Software Engineering:** The Software Engineering Discipline – Evolution and Impact – Software Development projects – Emergence of Software Engineering – Computer System Engineering – Software Life Cycle Models – classic Waterfall model – Iterative Lifecycle model – prototyping model – Evolutionary model – spiral model – Comparison of Life cycle models.

## UNIT – II

**Software Project Management and Requirements Analysis:** Responsibilities of a Software Project Manager – Project Planning – Metrics for Project Size Estimation – Empirical Estimation Techniques – COCOMO – Halstead's Software Science – Staffing Level Estimation – Scheduling – Organization and Team structures – Staffing – Risk Management – Software Configuration Management – Requirements Gathering and Analysis – Software Requirements specification – Formal System Specification – Axiomatic Specification - Algebraic Specification –4GL.

## UNIT – III

**Software Design and Function Oriented Software Design:** Outcome of a Design Process – Characteristics of a Good Software Design – Coupling and Cohesion – Approaches to Software Design – Object Oriented Vs Function Oriented Software Design approaches – Structured Analysis – Data Flow Diagrams – Applying DFD to Real time systems – Structured and Detailed Design.

## UNIT – IV

**Object Modelling and Object Oriented Software development:** Overview of OO concepts – UML – Use case model – Class diagrams – Interaction diagrams – Activity diagrams – state chart diagrams - Patterns – Types – Object Oriented Analysis and Design methodology – Interaction Modelling – OOD Goodness criteria.

## UNIT – V

**User Interface Design and Testing:** Characteristics of a good User Interface – Types – Fundamentals of Component based GUI Development – A User Interface Design methodology – Coding – Software Documentation – Testing – Unit Testing – Black Box testing – White Box testing – Debugging – Program Analysis tools – Integration testing – Testing Object Oriented programs – System Testing –Issues.

## TEXT BOOK

1. Rajib Mall, “Fundamentals of Software Engineering”, PHI Learning, Third Edition, 2013.

## REFERENCE BOOKS

1. Roger S. Pressman, “Software Engineering: A Practitioner's Approach”, McGraw-Hill International Edition, Seventh edition, 2009.
2. S. L. Pfleeger and J.M. Atlee, “Software Engineering Theory and Practice”, Pearson Education, Third edition, 2008.
3. Pankaj Jalote, “An Integrated Approach to Software Engineering”, Narosa, Third edition, 2008.
4. Ian Sommerville, “Software Engineering”, Pearson Education, Eighth edition, 2008

## DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**Subject: SOFTWARE ENGINEERING**

**Subject Code: CST55**

### **UNIT – I**

**Introduction to Software Engineering:** The Software Engineering Discipline – Evolution and Impact – Software Development projects – Emergence of Software Engineering – Computer System Engineering – Software Life Cycle Models – classic Waterfall model – Iterative Lifecycle model – prototyping model – Evolutionary model – spiral model – Comparison of Life cycle models

**1. Define build and fix style or Exploratory style**

In a Build and Fix approach, developers develop software quickly without making plan, specification and design. The different imperfections that are subsequently noticed while using or testing are fixed.

**2. Difference between programs and products**

| Programs                                                                                                                                                                                                                                          | Products                                                                                                                                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Usually small in size</li><li>• Author himself is sole user</li><li>• Single developer</li><li>• Lacks proper user interface</li><li>• Lacks proper documentation</li><li>• Ad hoc development.</li></ul> | <ul style="list-style-type: none"><li>• Large</li><li>• Large number of users</li><li>• Team of developers</li><li>• Well-designed interface</li><li>• Well documented &amp; user-manual prepared</li><li>• Systematic development</li></ul> |

**3. What is data flow oriented technique?**

The data flow-oriented technique advocate that the major data items handled by a system must first be identified and then the processing required on these data items to produce the desired should be determined.

**4. What is computer system engineering**

**Computer (Systems) Engineering** enables students to engage in the design of integrated hardware and software solutions for technical problems. They are involved in many aspects of computing, from the design of individual microprocessors, personal **computers**, and supercomputers, to circuit design.

**5. Write roles of structured analysis and structured design activity?**

- During Structured analysis, the detailed structure of the problem is examined.
- During Structured design, the results of structured analysis are transformed into the software design.

**6. What is phase containment of error?**

An engineering principle, which is used to detect errors as close to their points of introduction as possible is known as phase containment of errors.

**7. Compare different lifecycle models?**

**a. Iterative waterfall model**

- i. most widely used model.
- ii. But, suitable only for well-understood problems.

**b. Prototype model is suitable for projects not well understood:**

- i. user requirements
- ii. technical aspects

**c. Evolutionary model is suitable for large problems:**

- i. can be decomposed into a set of modules that can be incrementally implemented,
- ii. Incremental delivery of the system is acceptable to the customer.

**d. The spiral model:**

Suitable for development of technically challenging software products that are subject to several kinds of risks.

**8. List disadvantage of iterative waterfall model?**

- Each phase of an iteration is rigid with no overlaps
- Costly system architecture or design issues may arise because not all requirements are gathered up front for the entire lifecycle

**9. List disadvantage of classical waterfall model?**

- No scope for error correction
- Not allow for much reflection or revision
- Not well-documented or thought upon in the concept stage.

**10. What is classical waterfall model?**

A linear-sequential life cycle model, in which each phase must be completed before the next phase can begin and there is no overlapping in the phases.

**11. What is iterative water fall model?**

- A software development model, which is used to provide feedback paths from every phase to its preceding phase.
- The feedback paths allow for correction of the errors committed during a phase, as and when these are detected in a later phase.
- No feedback path to feasibility study from software requirements analysis and specification phase.

**12. What is Prototype model? (NOV 2018)**

- The prototyping model requires that **before carrying out the development of the actual software, a working prototype** of the system should be built.
- The Software Prototyping refers to building software application prototypes which display the functionality of the product under development but may not actually hold the exact logic of the original software.

**13. What is spiral model?**

Over each loop, one or more features of the product are elaborated and analyzed and the risks at that point of time are identified and are resolved through prototyping. Based on this, the identified features are implemented.

**14. What is evolutionary model (or) incremental model (or) successive versions model?**

In this model, first a simple working system is built, which subsequently undergoes many functionality improvement and additions until the desired systems is realized.

**15. What is software engineering? (NOV 2018)**

A Systematic and cost-effective technique to software development. These techniques have resulted from innovations as well as lessons learnt from past mistakes. It helps develop software using engineering approach.

**16. List types of system testing?**

- **$\alpha$ -testing:** The system testing performed by the development team
- **$\beta$ -testing:** This testing performed by a friendly set of customers
- **Acceptance testing:** This testing performed by the customer himself after the product delivery to determine whether to accept the delivered product or to reject it.

**17. List activities involved in maintenance?**

- Corrective maintenance: Correct errors which were not discovered during the product development phases.
- Perfective maintenance: Improve implementation of the system Enhance functionalities of the system.
- Adaptive maintenance: Port software to a new environment,
  - e.g. to a new computer or to a new operating system.

**18. What is SDLC (or) life cycle model?**

- A life cycle model prescribes the different activities that need to be carried out to develop a software product and the sequencing of these activities.
- A software life cycle is a series of identifiable stages that a software product undergoes during its lifetime.
- A software life cycle model is a descriptive and diagrammatic representation of the software life cycle.

**19. Define Software Engineering. (Nov 2017)**

"The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software"—IEEE Standard Glossary of Software Engineering Terminology.

**20. Which of the software engineering paradigms would be most effective? Why? (Nov 2017)**

Spiral model would be more effective because one or more features of the product are elaborated and analyzed and the risks at that point of time are identified and are resolved through prototyping.

**11 Marks:**

**1. Explain about Software Engineering?**

Computers are commercially being used for over the past sixty years. If we examine the way computers have evolved over this period, we can see that in the early days computers were very slow and lacked sophistication. The computational power and sophistication of computers have increased ever since, while their prices have dropped dramatically. The improvements to their speed and reductions to their cost were brought about by several technological breakthroughs that occurred at regular intervals.

The more powerful a computer is, the more sophisticated programs it can run. Therefore, with the every increase in capabilities of computers, software engineers have been called upon to solve larger and more complex problems, and that too in cost-effective and efficient ways. Software engineers have admirably coped with this challenge by innovating and by building upon their past programming experience. All these innovations and experiences have given rise to the discipline of software engineering. Let us now examine the scope of the software engineering discipline more closely.

**What is software engineering?** The essence of all past programming experiences and innovations for writing good quality programs in cost-effective and efficient ways have been systematically organized into a body of knowledge. This knowledge forms the foundation of the software engineering principles. From this point of view, we can define the scope of software engineering as follows.

The past experiences are systematically arranged and theoretical basis for them are provided wherever possible. Whenever no reasonable theoretical justification could be provided, the past experiences are adopted as rules of thumb. In contrast to the approach adopted by engineering disciplines, only exact solutions are accepted by science and that too when backed by rigorous proofs.

- In engineering disciplines, while designing a system, several conflicting goals might have to be optimized. In such situations, no unique solution may exist and several alternate solutions may be proposed. While selecting an appropriate solution out of several candidate solutions, various trade-offs are made based on issues such as cost, maintainability and usability. Therefore, to arrive at the final solution, several iterations and backtracking may have to be performed. In science, on the other hand, only unique solutions are possible.

**Why is software engineering neither a form of science nor an art?** Let us now analyze why software engineering discipline was not classified either as a form of science or an art. There exist several fundamental issues that set software engineering (and other engineering disciplines such as civil engineering) apart, from both science and art. Some of these important issues are the following:

- In an engineering discipline, a pragmatic approach to cost-effectiveness is adopted and economic concerns are addressed. Science normally does not concern itself with practical issues such as cost, maintainability and usability implications of a solution.
- Engineering disciplines are based on well-understood and quantitative principles. Art, on the other hand, is often based on subjective judgement which are based on qualitative attributes.

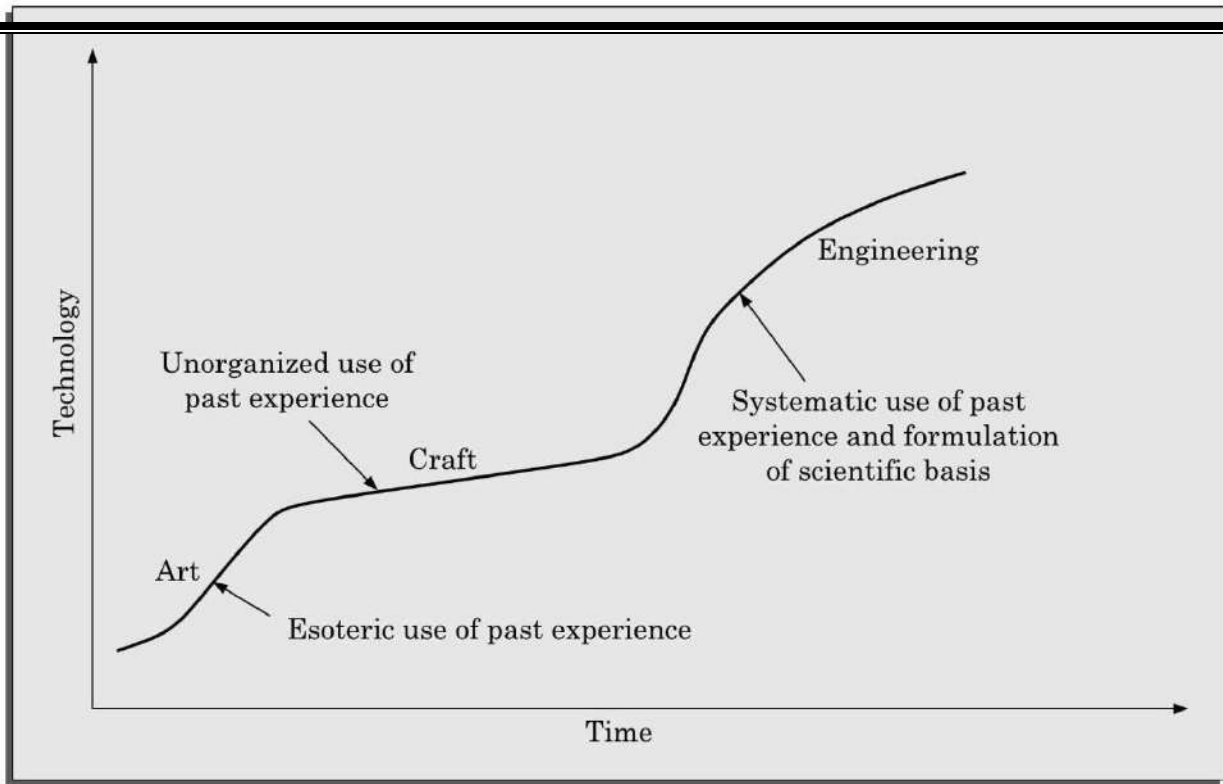
## 2. Describe Software Engineering Discipline- Evolution and Impact

### Evolution of an Art into an Engineering Discipline

The exploratory programming style is a very informal style of program development approach, and there are no set rules or recommendations that one has to adhere to — every programmer himself evolves his own software development techniques solely guided by his intuition, experience, whims and fancies. The exploratory style is also the one that is normally adopted by all students and novice programmers who do not have any exposure to software engineering principles. (We shall subsequently see that such a programming style results in poor quality and unmaintainable code and also makes program development very expensive and time-consuming.) We can consider the exploratory program development style as an art—since, any art is mostly guided by the intuition.

There are many stories about programmers in the past who were like proficient artists as a family secret. Slowly, technology graduated from an art to a craft form where tradesmen shared their knowledge with their apprentices, and the knowledge pool continued to grow. Much later, through a systematic organization of knowledge, and incorporation of scientific basis, modern steel making technology emerged. The story of the evolution of the software engineering discipline is not much different. In the early days of programming, there were good programmers and bad programmers. The good programmers knew certain principles (or *tricks*) that they seldom shared with the bad programmers. Over the years, all such good principles (or *tricks*) along with research innovations have systematically been organized into a body of knowledge that forms the discipline of software engineering.

seen from Figure 1.1 that every technology initially starts as a form of art. Over time, it graduates to a craft, and finally emerges as an engineering discipline. Let us illustrate this fact using an example. Consider the evolution of the iron making technology. In ancient times, only a few people knew how to make iron. Those who knew iron making, kept it a closely-guarded secret. This esoteric knowledge got transferred from generation to generation



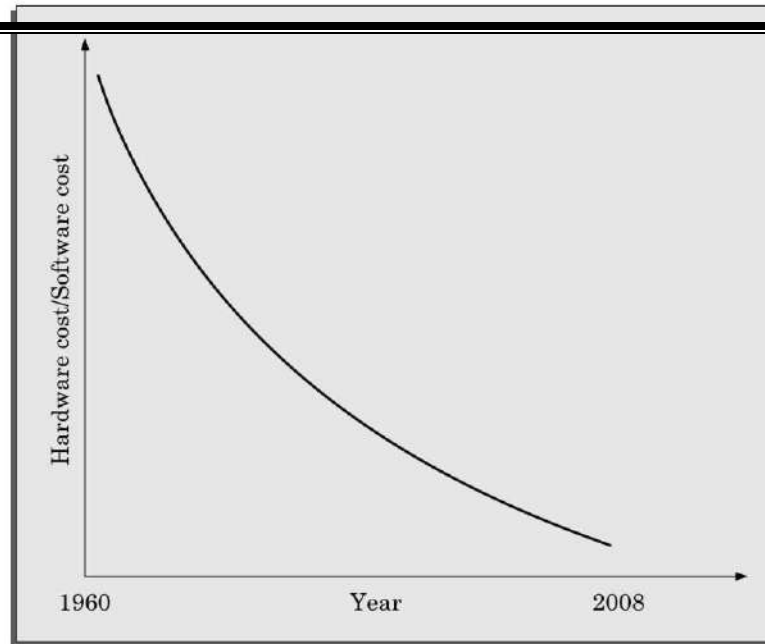
Evolution of technology with time

Software engineering principles are now being widely used in industries, and new principles are still continuing to emerge at a very rapid rate making this discipline highly dynamic. In spite of its wide acceptance, critics point out that many of the methodologies and guidelines provided by the software engineering discipline lack scientific basis, are subjective, and are often inadequate. Yet, there is no denying the fact that adopting software engineering techniques facilitates development of high quality software in a cost-effective and timely manner. Software engineering practices have proven to be indispensable to the development of large software products—though exploratory styles can often be used successfully to develop small programs.

## CAUSES AND SOLUTIONS FOR SOFTWARE CRISIS

Software engineering appears to be among the few options available to tackle the present software crisis.

To explain the present software crisis in simple words, consider the following. The expenses that organizations all around the world are incurring on software purchases compared to those on hardware purchases have been showing a worrying trend over the years (as shown in fig. )



Change in the relative cost of hardware and software over time

Organizations are spending larger and larger portions of their budget on software. Not only are the software products turning out to be more expensive than hardware, but they also present a host of other problems to the customers: software products are difficult to alter, debug, and enhance; use resources non-optimally; often fail to meet the user requirements; are far from being reliable; frequently crash; and are often delivered late. Among these, the trend of increasing software costs is probably the most important symptom of the present software crisis. Remember that the cost we are talking of here is not on account of increased features, but due to ineffective development of the product characterized by inefficient resource usage, and time and cost over-runs.

There are many factors that have contributed to the making of the present software crisis. Factors are

- Larger problem sizes,
- Lack of adequate training in software engineering,
- Increasing skill shortage,
- Low productivity improvements.

It is believed that the only satisfactory solution to the present software crisis can possibly come from a spread of software engineering practices among the engineers, coupled with further advancements to the software engineering discipline itself.

### 3. Explain Software Development Projects

#### Program vs. software product

- Programs are developed by individuals for their personal use. They are therefore, small in size and have limited functionality but software products are extremely large.
- In case of a program, the programmer himself is the sole user but on the other hand, in case of a software product, most users are not involved with the development.



- In case of a program, a single developer is involved but in case of a software product, a large number of developers are involved. For a program, the user interface may not be very important, because the programmer is the sole user. On the other hand, for a software product, user interface must be carefully designed and implemented because developers of that product and users of that product are totally different.
- In case of a program, very little documentation is expected, but a software product must be well documented. A program can be developed according to the programmer's individual style of development, but a software product must be developed using the accepted software engineering principles.

### **Types of Software Development**

- ☐ Software Product Development Project
- ☐ Outsourced Project

#### **Software Product Development Project**

- ☐ A Software product can be a generic or a customer development product.
- ☐ Generic products used by large and diverse range of customers.
- ☐ A customized software product is developed according to the specification drawn up by one or at most few customers

#### **Out Sourced Project**

- ☐ It can make good commercial sense for a company developing a software product to outsource some part of its work as a project to another company.

### **Software Projects being undertaken by Indian Companies**

Indian software industries have excelled in executing outsourced software projects and have made a name for themselves all over the world. Of late, the Indian companies have slowly started to focus on product development as well (which was the front they were lacking in till date).

The types of development work being handled by a company can have an impact on its profitability. For example, a company that has developed a generic software product usually gets an uninterrupted stream of revenue over several years. On the other hand, outsourced projects fetch a one time revenue to the developing company.

## **4. EMERGENCE OF SOFTWARE ENGINEERING**

### **Early Computer Programming (1950s):**

- Programs were being written in assembly language.
- Programs were limited to about a few hundreds of lines of assembly code.
- Every programmer developed his own style of writing programs:
- According to his intuition (exploratory programming).

## High-Level Language Programming (Early 60s)

- ☐ High-level languages such as FORTRAN, ALGOL, and COBOL were introduced:
- ☐ This reduced software development efforts greatly
- ☐ Software development style was still exploratory.
- ☐ Typical program sizes were limited to a few thousands of lines of source code

### Control Flow-Based Design (late 60s)

- ☐ Size and complexity of programs increased further: Exploratory programming style proved to be insufficient.
- ☐ Programmers found very difficult to write cost-effective and correct programs
- ☐ Programs written by others very difficult to understand and maintain.
  - To cope up with this problem, experienced programmers advised: ``Pay particular attention to the design of the program's control structure.''
- ☐ A program's control structure indicates: the sequence in which the program's instructions are executed.
- ☐ To help design programs having good control structure: flow charting technique was developed.
- ☐ Using flow charting technique: one can represent and design a program's control structure.
- ☐ Usually one understands a program: by mentally simulating the program's execution sequence.
- ☐ A program having a messy flow chart representation: difficult to understand and debug.
- ☐ It was found: GO TO statements makes control structure of a program messy, GO TO statements alter the flow of control arbitrarily.
- ☐ The need to restrict use of GO TO statements was recognized.
- ☐ Many programmers had extensively used assembly languages. JUMP instructions are frequently used for program branching in assembly languages, programmers considered use of GO TO statements inevitable.
  - At that time, Dijkstra published his article: "Go to Statement Considered Harmful" Comm. of ACM, 1969. Many programmers were unhappy to read his article.
- ☐ They published several counter articles: highlighting the advantages and inevitability of GO TO statements.
- ☐ But, soon it was conclusively proved: only three programming constructs are sufficient to

express any programming logic:

- Sequence (e.g. a=0; b=5;)
  - Selection (e.g. if(c=true) k=5 else m=5;)
  - Iteration (e.g. while (k>0) k=j-k;)
- ☐ Everyone accepted: it is possible to solve any programming problem without using GO TO statements. This formed the basis of Structured Programming methodology.

### **Structured Programming**

- ☐ A program is called structured when it uses only the following types of constructs:
- sequence,
  - selection,
  - iteration
- ☐ Unstructured control flows are avoided.
- ☐ Consist of a neat set of modules.
- ☐ Use single-entry, single-exit program constructs.
- ☐ However, violations to this feature are permitted: due to practical considerations such as: premature loop exit to support exception handling.
- ☐ Structured programs are: Easier to read and understand, easier to maintain, require less effort and time for development.
- ☐ Research experience shows programmers commit less number of errors
- while using structured if-then-else and do-while statements
  - Compared to test-and-branch constructs.

### **Data Structure-Oriented Design (Early 70s)**

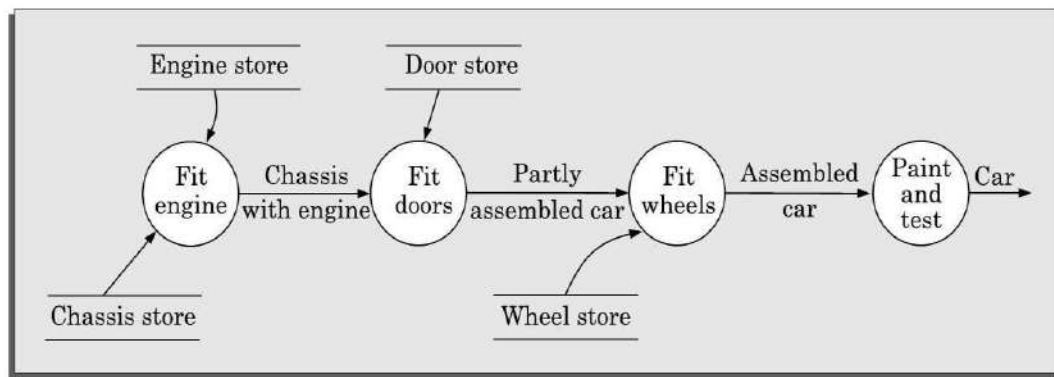
- ☐ Soon it was discovered it is important to pay more attention to the design of data structures of a program than to the design of its control structure.
- ☐ Techniques which emphasize designing the data structure derive program structure from it: are called data structure-oriented design techniques
- ☐ Example of data structure-oriented design technique: Jackson's Structured Programming (JSP) methodology developed by Michael Jackson in 1970s.
- ☐ JSP technique: program code structure should correspond to the data structure
- ☐ In JSP methodology: a program's data structures are first designed using notations for Sequence, selection, and iteration
- ☐ Then data structure design is used: to derive the program structure.

- Several other data structure-oriented Methodologies also exist: e.g., Warnier-Orr Methodology.

### Data Flow-Oriented Design (Late 70s)

- Data flow-oriented techniques advocate: the data items input to a system must first be identified, processing required on the data items to produce the required outputs should be determined.
- Data flow technique identifies: different processing stations (functions) in a system the items (data) that flow between processing stations.
- Data flow technique is a generic technique: can be used to model the working of any system not just software systems. A major advantage of the data flow technique is its simplicity.

### Data Flow Model of a Car Assembly Unit

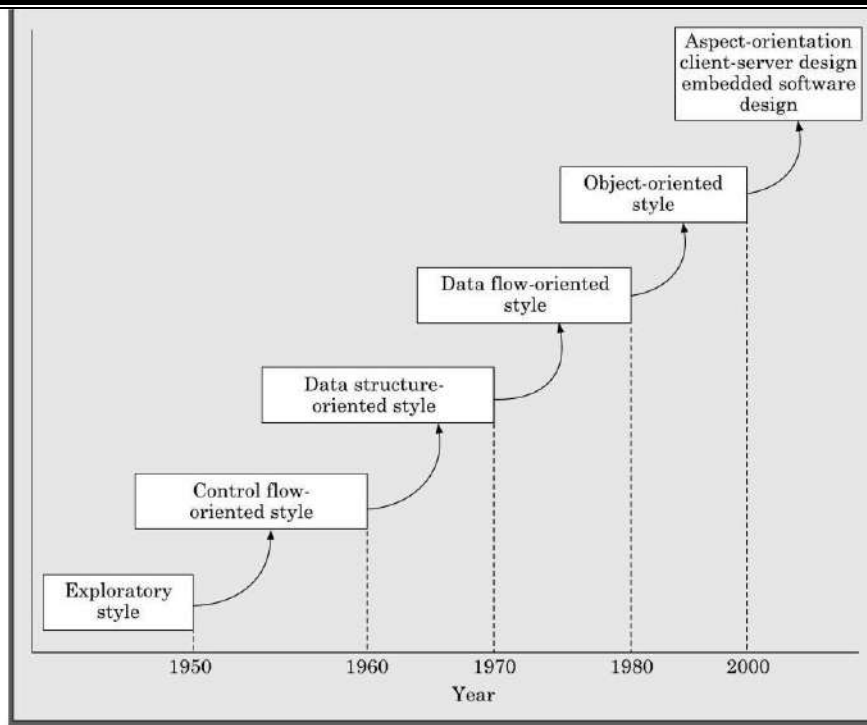


### Data Flow Model of a Car Assembly

### Unit Object-Oriented Design (80s)

- Object-oriented technique: an intuitively appealing design approach, natural objects (such as employees, pay-roll-register, etc.) occurring in a problem are first identified.
- Relationships among objects: such as composition, reference, and inheritance are determined.
- Each object essentially acts as a data hiding (or data abstraction) entity.
- Object-Oriented Techniques have gained wide acceptance:
  - Simplicity
  - Reuse possibilities
  - Lower development time and cost
  - More robust code
  - Easy maintenance

### Evolution of Design Techniques



### Evolution of software design

#### technique Evolution of Other Software Engineering

##### Techniques

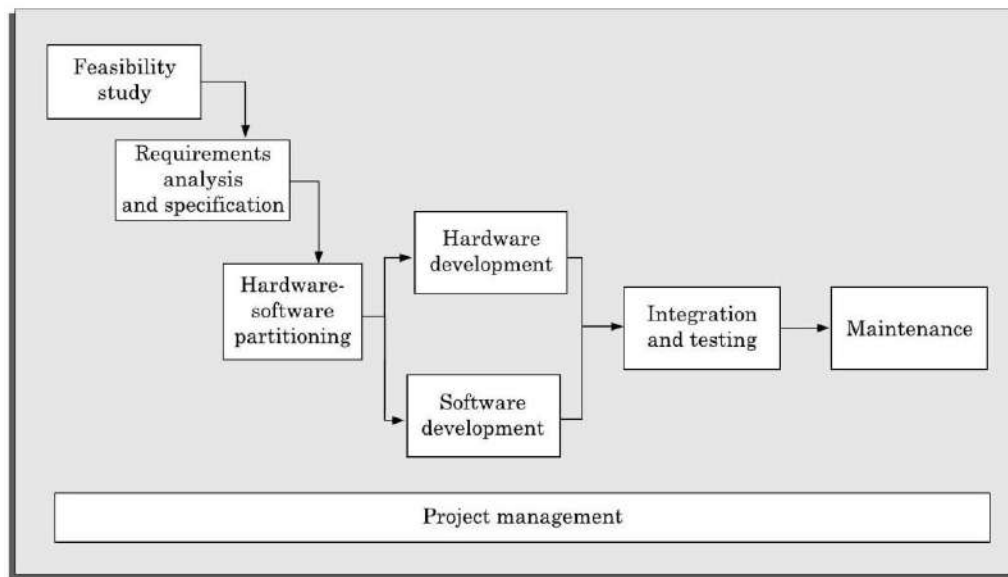
- The improvements to the software design methodologies are indeed very conspicuous.
- In additions to the software design techniques: several other techniques evolved
  - life cycle models,
  - specification techniques,
  - project management techniques,
  - testing techniques,
  - debugging techniques,
  - quality assurance techniques,
  - software measurement techniques,
  - CASE tools, etc.

#### 5. Differences between the exploratory style and modern software development practices

- Use of Life Cycle Models
- Software is developed through several well-defined stages:
  - requirements analysis and specification,
  - design,
  - coding,
  - Testing, etc.
- Emphasis has shifted from error correction to error prevention.
- Modern practices emphasize: detection of errors as close to their point of introduction as possible.
- In exploratory style, errors are detected only during testing, coding is synonymous with program development
- Now, focus is on detecting as many errors as possible in each phase of development, coding is considered only a small part of program development effort.
- A lot of effort and attention is now being paid to: requirements specification.
- Also, now there is a distinct design phase: standard design techniques are being used.
- During all stages of development process: Periodic reviews are being carried out
- Software testing has become systematic: standard testing techniques are available.
- There is better visibility of design and code: visibility means production of good quality, consistent and standard documents. In the past, very little attention was being given to producing good quality and consistent documents. We will see later that increased visibility makes software project management easier
- Because of good documentation: fault diagnosis and maintenance are smoother now.
- Several metrics are being used: help in software project management, quality assurance, etc.
- Projects are being thoroughly planned:
  - estimation,
  - scheduling,
  - Monitoring mechanisms.
- Use of CASE tools

## 6. COMPUTER SYSTEM ENGINEERING

In all the discussions so far, we assumed that the software being developed would run on some general-purpose hardware platform such as a desktop computer. But, in several situations it may be necessary to develop special hardware on which the software would run. Examples of such systems are numerous. Examples of such software include a robot, a factory automation system, a cell phone, etc. In a cell phone, there is a special processor and other specialized devices such as a speaker and a microphone. It can run only the programs written specifically for it. Development of such systems entails development of both software and specific hardware that would run the software. Computer systems engineering addresses development of such systems requiring development of both software and specific hardware to run the software. Thus, systems engineering encompasses software engineering (see Figure 1.12).



## Computer System Engineering

The general model of systems engineering is shown schematically in Figure 1.12. One of the important stages in systems engineering is the stage in which decision is made regarding the parts of the problems that are to be implemented in hardware and the ones that would be implemented in software. This has been represented by the box captioned hardware-software partitioning in Figure 1.12. While partitioning the functions between hardware and software, several trade-offs such as flexibility, cost, speed of operation, etc., need to be considered. The functionality implemented in hardware run faster. On the other hand, functionalities implemented in software can be more easily be extended. Also, it is difficult to implement complex functions in hardware. Also, functions implemented in hardware incur extra space, weight, manufacturing cost, and power overhead.

After the hardware-software partitioning stage, development of hardware and software are carried out concurrently. In systems engineering, testing the software during development becomes a tricky issue, the hardware on which the software would run and tested would still be under development — remember that the hardware and the software are being developed at the same time. To test the software during development, it usually becomes necessary to develop simulators that mimic the features of the hardware being developed. The software is tested using these simulators. Once both hardware and software development are complete, these are integrated and tested. The project management activity is required throughout the duration of system development (as shown in the figure. In this book, we have confined our attention to software engineering only.

## 7. SOFTWARE LIFE CYCLE

## ☐ Software life cycle (or software process):

- series of identifiable stages that a software product undergoes during its lifetime:

- Feasibility study
- requirements analysis and specification,
- design,
- coding,
- testing
- Maintenance.

### **LIFE CYCLE MODEL**

## ☐ A software life cycle model (or process model):

- a descriptive and diagrammatic model of software lifecycle:
- identifies all the activities required for product development,
- establishes a precedence ordering among the different activities,
- Divides life cycle into phases.

## ☐ Several different activities may be carried out in each life cycle phase.

- For example, the design stage might consist of:
  - structured analysis activity followed by
  - Structured design activity.

### **Why Model Life Cycle?**

- ☐ A written description: forms a common understanding of activities among the software developers.
- ☐ Helps in identifying inconsistencies, redundancies, and omissions in the development process.
- ☐ Helps in tailoring a process model for specific projects.

## ☐ Processes are tailored for special projects.

- A documented process model helps to identify where the tailoring is to occur.

The development team must identify a suitable life cycle model: and then adhere to it.  
Primary advantage of adhering to a life cycle model: helps development of software in a systematic and disciplined manner

## ☐ When a program is developed by a single programmer

- He has the freedom to decide his exact steps.

## ☐ When a software product is being developed by a team:

- There must be a precise understanding among team members as to when to do what,



➤ Otherwise it would lead to chaos and project failure.

- A software project will never succeed if:
  - one engineer starts writing code,
  - another concentrates on writing the test document first,
  - yet another engineer first defines the file structure
  - Another defines the I/O for his portion first.

### **Phase Entry and Exit Criteria**

A life cycle model defines entry and exit criteria for every phase. A phase is considered to be complete: Only when all its exit criteria are satisfied.

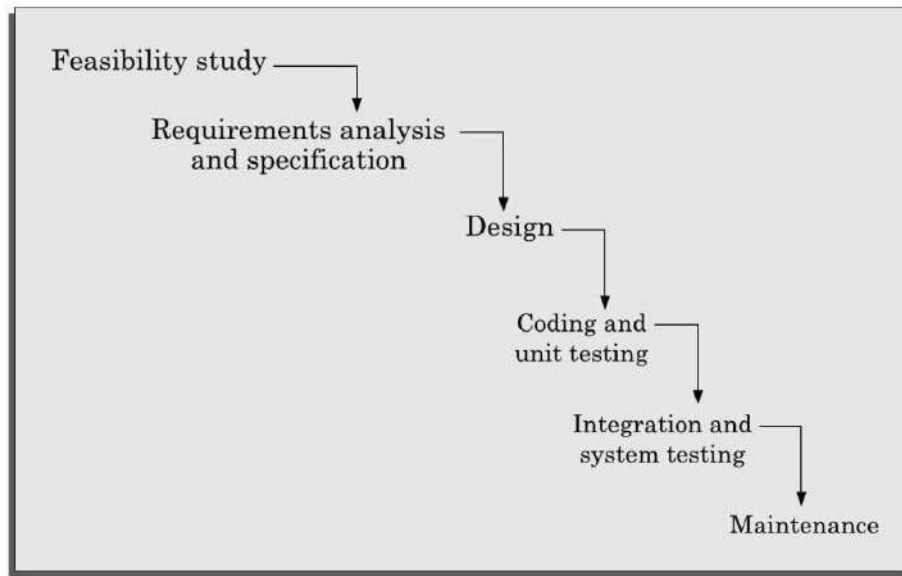
- The phase exit criteria for the software requirements specification phase: Software Requirements Specification (SRS) document is complete, reviewed, and approved by the customer.
- A phase can start: only if its phase-entry criteria have been satisfied.
- It becomes easier for software project managers: to monitor the progress of the project.
- When a life cycle model is adhered to, the project manager can at any time fairly accurately tell, at which stage (e.g., design, code, test, etc.) of the project is. Otherwise, it becomes very difficult to track the progress of the project the project manager would have to depend on the guesses of the team members.
- This usually leads to a problem: known as the 99% complete syndrome.
- Many life cycle models have been proposed. We will confine our attention to a few important and commonly used models.
  - Classical waterfall model
  - Iterative waterfall,
  - Evolutionary,
  - Prototyping, and
  - Spiral model

## **8. CLASSICAL WATERFALL MODEL (NOV 2018)**

- **Classical waterfall model divides life cycle into phases:**

➤ Feasibility study.

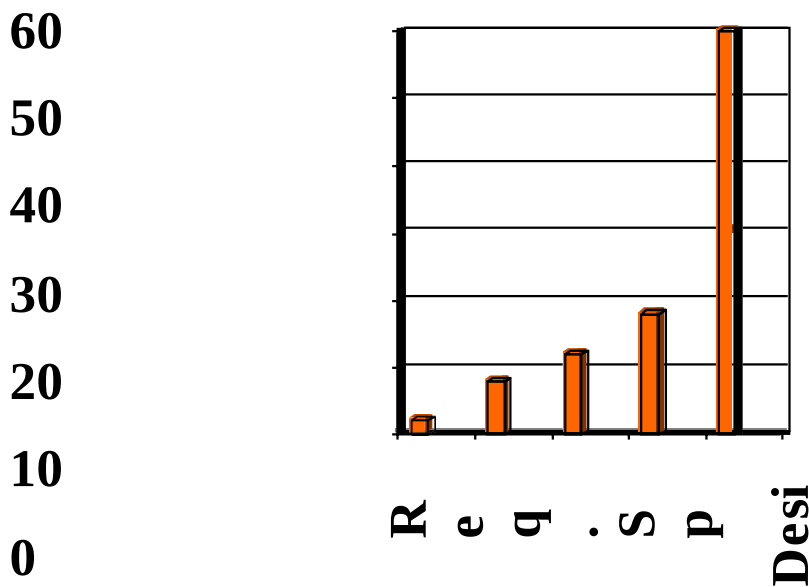
- Requirements analysis and specification,
- Design,
- Coding and unit testing,
- Integration and system testing,
- Maintenance.



### Classical waterfall model

#### Phases of Classical Waterfall Model

- ☐ Phases between feasibility study and testing known as development phases.
- ☐ Among all life cycle phases maintenance phase consumes maximum effort.
- ☐ Among development phases, testing phase consumes the maximum effort.



#### Relatively effort distribution among different phases of a typical product

- ☐ Most organizations usually define:
  - standards on the outputs (deliverables) produced at the end of every phase

- entry and exit criteria for every phase.
- They also prescribe specific methodologies for:
  - Specification,
  - Design,
  - Testing,
  - Project management, etc.
- The guidelines and methodologies of an organization: called the organization's software development methodology.
- Software development organizations: expect fresh engineers to master the organization's software development methodology.

### **Feasibility Study**

- Main aim of feasibility study: determine whether developing the product
  - Financially worthwhile
  - Technically feasible.
- First roughly understand what the customer wants:
  - Different data which would be input to the system,
  - Processing needed on these data,
  - Output data to be produced by the system,
  - Various constraints on the behavior of the system

### **Activities during Feasibility Study**

- Work out an overall understanding of the problem.
- Formulate different solution strategies.
- Examine alternate solution strategies in terms of:
  - Resources required,
  - Cost of development, and
  - Development time.
- Perform a cost/benefit analysis: to determine which solution is the best. you may determine that none of the solutions is feasible due to:
  - High cost,
  - Resource constraints,
  - Technical reasons.

### **Requirements Analysis and Specification**

- Aim of this phase:
  - Understand the exact requirements of the customer,
  - Document them properly.

- Consists of two distinct activities:
  - Requirements gathering and analysis
  - Requirements specification.

### **Goals of Requirements Analysis**

- Collect all related data from the customer:
  - analyze the collected data to clearly understand what the customer wants,
  - find out any inconsistencies and incompleteness in the requirements,
  - Resolve all inconsistencies and incompleteness.

### **Requirements Gathering**

- Gathering relevant data: usually collected from the end-users through interviews and discussions. For example, for a business accounting software: interview all the accountants of the organization to find out their requirements.

### **Requirements Analysis**

- The data you initially collect from the users: would usually contain several contradictions and ambiguities, each user typically has only a partial and incomplete view of the system.
- Ambiguities and contradictions must be identified, resolved by discussions with the customers.
- Next, requirements are organized into a Software Requirements Specification (SRS) document
- Engineers doing requirements analysis and specification are designated as analysts

### **Design**

- Design phase transforms requirements specification into a form suitable for implementation in some programming language.
- In technical terms during design phase, software architecture is derived from the SRS document.
- Two design approaches:
  - Traditional approach,
  - Object oriented approach.

### **Traditional Design Approach**

- Consists of two activities:
  - Structured analysis
  - Structured design

### **Structured Analysis Activity**

- Identify all the functions to be performed.
- Identify data flow among the functions.
- Decompose each function recursively into sub-functions. Identify data flow among the sub-functions as well.
- Carried out using Data flow diagrams (DFDs).
- After structured analysis, carry out structured design: architectural design (or high-level design), detailed design (or low-level design).

#### Structured Design

- High-level design:
  - Decompose the system into *modules*,
  - Represent invocation relationships among the modules.
- Detailed design:
  - Different modules designed in greater detail data structures and algorithms for each module are designed.

#### Object Oriented Design Approach

- First identify various objects (real world entities) occurring in the problem identify the relationships among the objects. For example, the objects in a pay-roll software maybe:
  - employees,
  - managers,
  - pay-roll register,
  - Departments, etc.
- Object structure
  - Further refined to obtain the detailed design.
- OOD has several advantages:
  - Lower development effort,
  - Lower development time,
  - Better maintainability.

#### Coding and Unit Testing(Implementation)

- Purpose of implementation phase (aka coding and unit testing phase): translate software design into source code.
- During the implementation phase each module of the design is coded, each module is unit tested, tested independently as a standalone unit, and debugged, each module is documented.
- The purpose of unit testing: test if individual modules work correctly.
- The end product of implementation phase: a set of program modules that have been tested individually.

#### Integration and System Testing

- Different modules are integrated in a planned manner: modules are almost never integrated in one shot. Normally integration is carried out through a number of steps.
- During each integration step, the partially integrated system is tested.



### System Testing

- ☐ After all the modules have been successfully integrated and tested system testing is carried out.
- ☐ Goal of system testing ensure that the developed system functions according to its requirements as specified in the SRS document.
- ☐ System testing usually consists of three different kinds of testing activities
  - 1.  $\alpha$ -testing:**  $\alpha$  testing is the system testing performed by the development team
  - 2.  $\beta$ -testing:** This is the system testing performed by a friendly set of customers
  - 3. Acceptance testing:** This is the system testing performed by the customer himself after the product delivery to determine whether to accept the delivered product or to reject it.

### Maintenance

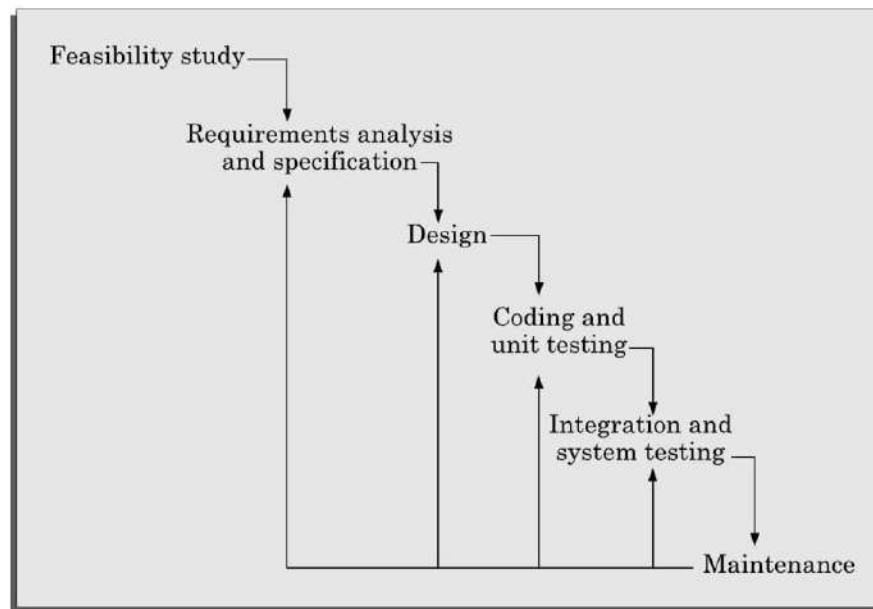
- ☐ **Maintenance of any software product:**
  - Requires much more effort than the effort to develop the product itself.
  - Development effort to maintenance effort is typically 40:60.
- ☐ Corrective maintenance:
  - Correct errors which were not discovered during the product development phases.
- ☐ Perfective maintenance:
  - Improve implementation of the system
  - Enhance functionalities of the system.
- ☐ Adaptive maintenance:
  - Port software to a new environment, e.g. to a new computer or to a new operating system

## 9. ITERATIVE WATERFALL MODEL

- ☐ Classical waterfall model is idealistic assumes that no defect is introduced during any

development activity. in practice defects do get introduced in almost every phase of the lifecycle.

- ❑ Defects usually get detected much later in the life cycle: For example, a design defect might go unnoticed till the coding or testing phase.
- ❑ Once a defect is detected: we need to go back to the phase where it was introduced redo some of the work done during that and all subsequent phases.
- ❑ Therefore we need feedback paths in the classical waterfall model



**Iterative waterfall model**

- ❑ Errors should be detected in the same phase in which they are introduced.
- ❑ For example:
  - if a design problem is detected in the design phase itself, the problem can be taken care of much more easily than say if it is identified at the end of the integration and system testing phase

#### Phase containment of errors

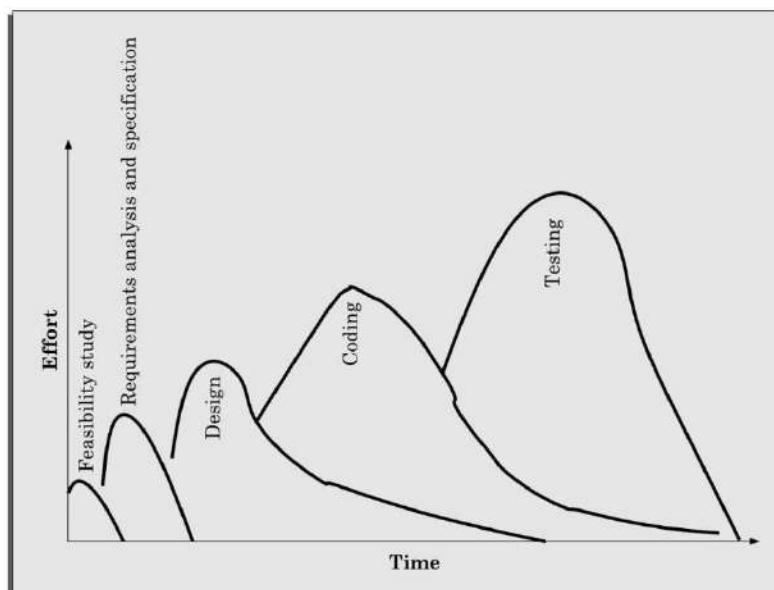
- ❑ Reason: rework must be carried out not only to the design but also to code and test phases.
- ❑ The principle of detecting errors as close to its point of introduction as possible: is known as phase containment of errors.
- ❑ Iterative waterfall model is by far the most widely used model. Almost every other model is derived from the waterfall model.

#### Shortcomings of the Iterative Waterfall Model

A good understanding of the waterfall model is necessary to appreciate and understand the other development processes. However, the waterfall model suffers from many shortfalls. These shortcomings are addressed to different extents by the other life cycle models that we shall discuss subsequently. Some of the glaring shortcomings of the waterfall model are the following:

1. The waterfall model cannot satisfactorily handle the different types of risks that a real-life software project may suffer from. For example, the waterfall model assumes that the requirements are completely specified before the rest of the development activities can start. Therefore, it cannot accommodate the uncertainties concerning the requirements that exist at the beginning of most of the projects. As a result, it cannot be satisfactorily used in projects where the customer is not clear about his requirements and can provide rough requirements only.
2. To achieve better efficiency and higher productivity, most real life projects find it difficult to follow the rigid phase sequence prescribed by the waterfall model. By the term *a rigid phase sequence*, we mean that a phase can start only after the previous phase is complete in all respects. A rigid adherence to the waterfall model would create *blocking states* in the system. That is, once a developer completes his work, he idles waiting for the phase to get over. Since the work required to be done in a phase is distributed among several team members, some members may complete their work earlier than other members.

The team members who complete their work early would not be able to their next activity and would have to idle for other members to complete their work. This would clearly be a wastage of resources and wastages are rarely tolerated in real projects. For example, if an engineer works fast and completes the design of the parts assigned to him early, he should not idle, waiting for the others to complete their designs. Instead, he should proceed to start the activities for the next phase. This situation would give rise to overlapping of phases as shown in Figure 2.4. Of course, as we had discussed the overlap of different phases can also be due to correction of errors, when backtracking to a previous phase may be necessary.



**Distribution of effort over time for various phases in the iterative waterfall model**

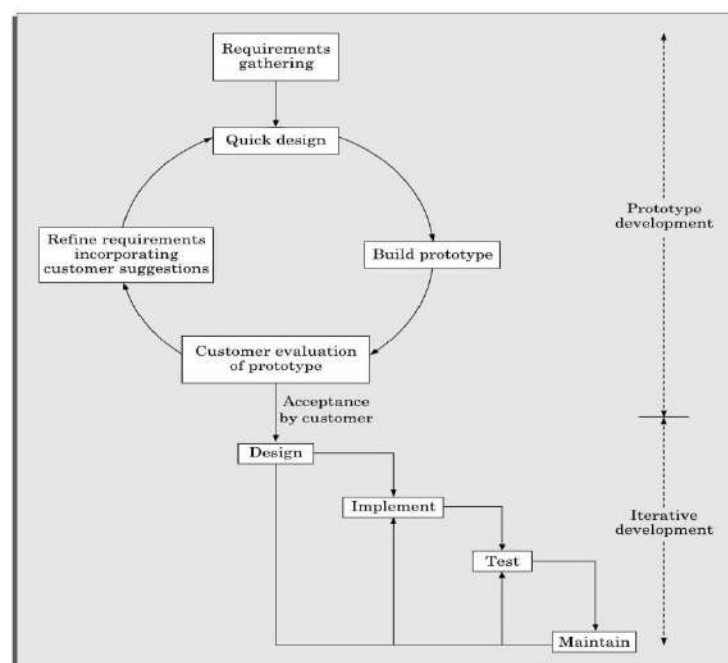
## **PROTOTYPING MODEL**



- ❑ Before starting actual development, a working prototype of the system should first be built.
- ❑ A prototype is a toy implementation of a system:
  - limited functional capabilities,
  - low reliability,
  - Inefficient performance.

#### Reasons for developing a prototype

- ❑ Illustrate to the customer input data formats, messages, reports, or interactive dialogs.
- ❑ Examine technical issues associated with product development: Often major design decisions depend on issues like:
  - response time of a hardware controller,
  - efficiency of a sorting algorithm, etc
- ❑ The third reason for developing a prototype is: it is impossible to "get it right" the first time, we must plan to throw away the first product if we want to develop a good product.
- ❑ Start with approximate requirements.
- ❑ Carry out a quick design.
- ❑ Prototype model is built using several short-cuts: Short-cuts might involve using inefficient, inaccurate, or dummy functions. A function may use a table look-up rather than performing the actual computations.
- ❑ The developed prototype is submitted to the customer for his evaluation: Based on the user feedback, requirements are refined. This cycle continues until the user approves the prototype.
- ❑ The actual system is developed using the classical waterfall approach.



Prototyping model of software development

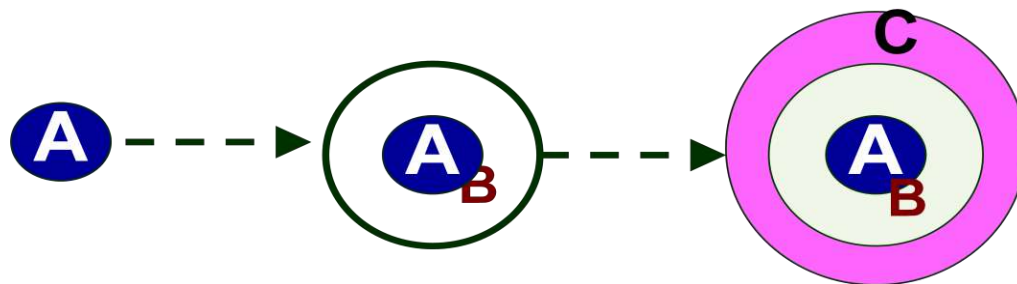
- ❑ Requirements analysis and specification phase becomes redundant: final working

prototype (with all user feedbacks incorporated) serves as an animated requirements specification.

- ❑ Design and code for the prototype is usually thrown away: However, the experience gathered from developing the prototype helps a great deal while developing the actual product.
- ❑ Even though construction of a working prototype model involves additional cost overall development cost might be lower for:
  - Systems with unclear user requirements,
  - Systems with unresolved technical issues.
- ❑ Many user requirements get properly defined and technical issues get resolved: these would have appeared later as change requests and resulted in incurring massive redesign costs

## 10. EVOLUTIONARY MODEL

- ❑ Evolutionary model (aka successive versions or incremental model):
  - The system is broken down into several modules which can be incrementally implemented and delivered.
- ❑ First develop the core modules of the system.
- ❑ The initial product skeleton is refined into increasing levels of capability: by adding new functionalities in successive versions.
- ❑ Successive version of the product:
  - Functioning systems capable of performing some useful work.
  - A new release may include new functionality; also existing functionality in the current release might have been enhanced.



### Advantages of Evolutionary Model

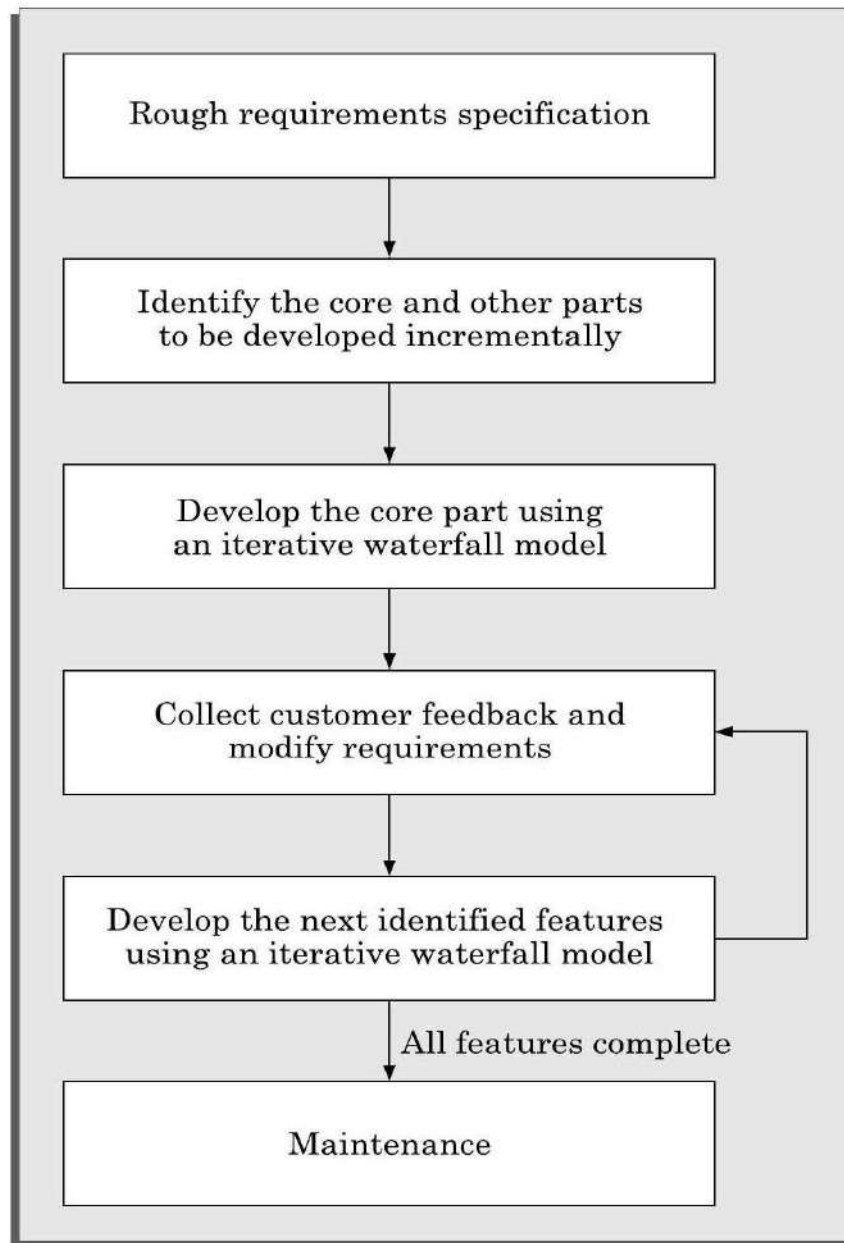
- ❑ Users get a chance to experiment with a partially developed system:
  - much before the full working version is released,
- ❑ Helps finding exact user requirements:

- Much before fully working system is developed.

- Core modules get tested thoroughly:
  - Reduces chances of errors in final product

### **Disadvantages of Evolutionary Model**

- Often, difficult to subdivide problems into functional units:
  - Which can be incrementally implemented and delivered
  - Evolutionary model is useful for very large problems, where it is easier to find modules for incremental implementation.

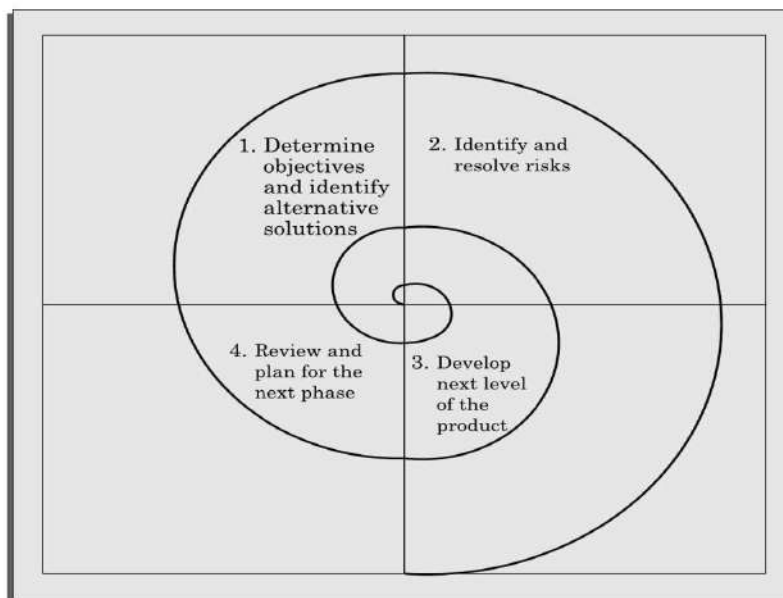


**Evolutionary model of software development**

- Many organizations use a combination of iterative and incremental development:
  - A new release may include new functionality
  - Existing functionality from the current release may also have been modified.
- **Several advantages:**
  - Training can start on an earlier release customer feedback taken into account
  - Markets can be created: for functionality that has never been offered.
  - Frequent releases allow developers to fix unanticipated problems quickly

## 11. SPIRAL MODEL (NOV 2018)

- Proposed by Boehm in 1988.
- Each loop of the spiral represents a phase of the software process:
  - The innermost loop might be concerned with system feasibility,
  - The next loop with system requirements definition,
  - The next one with system design, and soon.
- There are no fixed phases in this model; the phases shown in the figure are just examples.
- The team must decide: How to structure the project into phases.
- Start work using some generic model: add extra phases for specific projects or when problems are identified during a project.
- Each loop in the spiral is split into four sectors (quadrants).



**Spiral model of software development**

### Objective Setting (First Quadrant)

- ☐ Identify objectives of the phase,
- ☐ Examine the risks associated with these objectives.
  - Risk: any adverse circumstance that might hamper successful completion of a software project.
  - Find alternate solutions possible

### **Risk Assessment and Reduction (Second Quadrant)**

- ☐ For each identified project risk, a detailed analysis is carried out.
- ☐ Steps are taken to reduce the risk.
- ☐ For example, if there is a risk that the requirements are inappropriate: a prototype system may be developed.
- ☐ Development and Validation (Third quadrant): develop and validate the next level of the product.
- ☐ Review and Planning (Fourth quadrant): review the results achieved so far with the customer and plan the next iteration around the spiral.
- ☐ With each iteration around the spiral: progressively more complete version of the software gets built.

### **Spiral Model as a Meta model**

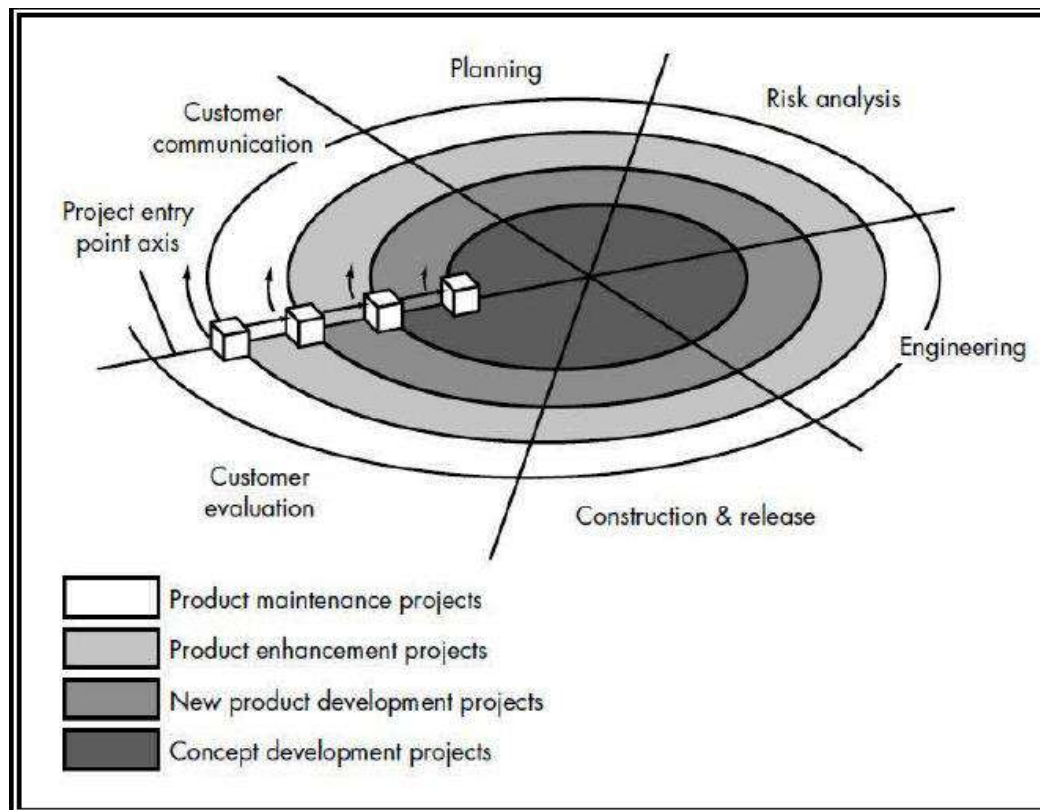
- ☐ Subsumes all discussed models:
  - A single loop spiral represents waterfall model.
  - Uses an evolutionary approach iterations through the spiral are evolutionary levels.
- Enables understanding and reacting to risks during each iteration along the spiral.
- Uses: prototyping as a risk reduction mechanism retains the step-wise approach of the waterfall model.

### **Comparison of Different Life Cycle Models**

- ☐ **Iterative waterfall model** most widely used model. But, suitable only for well-understood
- ☐ Problems.
- ☐ **Prototype model** is suitable for projects not well understood: user requirements technical aspects.
- ☐ **Evolutionary model is suitable for large problems:** can be decomposed into a set of modules that can be incrementally implemented, incremental delivery of the system is acceptable to the customer.
- ☐ **The spiral model:** suitable for development of technically challenging software products that are subject to several kinds of risks.

**12. a) Do you feel spiral model is a good enough model to be followed in industry? Justify your answer with two reasons. (6) (Nov 2017)**

- Spiral model is an evolutionary software process model that couples the iterative nature of prototyping with the controlled and systematic aspects of the linear sequential model.
- It provides the potential for rapid development of incremental versions of the software.
- Using the spiral model, software is developed in a series of incremental releases.
- A spiral model is divided into a number of framework activities, also called task regions.
- Typically, there are between three and six task regions as shown in the figure and all performs different tasks.



### Spiral Model

Spiral model contains six task regions:

1. **Customer communication** – tasks required to establish effective communication between developer and customer.
2. **Planning** – tasks required to define resources, timelines and other project related information.
3. **Risk analysis** - tasks required to assess both technical and management risks.
4. **Engineering** – tasks required to build one or more representations of the application.
5. **Construction and release** – tasks required to construct, test, install and provide user support(eg. documentation and training)

**12.b) What are the necessities of Life Cycle Model? Elaborate on the various issues of software. (5) (Nov 2017)**

- A written description: forms a common understanding of activities among the software developers.
- Helps in identifying inconsistencies, redundancies, and omissions in the development process.
- Helps in tailoring a process model for specific projects.
- Processes are tailored for special projects.
- A documented process model helps to identify where the tailoring is to occur.

The development team must identify a suitable life cycle model: and then adhere to it. Primary advantage of adhering to a life cycle model: helps development of software in a systematic and disciplined manner

- ☐ When a program is developed by a single programmer
  - He has the freedom to decide his exact steps.
- ☐ When a software product is being developed by a team:
  - There must be a precise understanding among team members as to when to do what,
  - Otherwise it would lead to chaos and project failure.
- ☐ A software project will never succeed if:
  - one engineer starts writing code,
  - another concentrates on writing the test document first,
  - yet another engineer first defines the file structure
  - Another defines the I/O for his portion first.

**13. Explain the various phases of software development life cycle (SDLC) and identify deliverable at each phases? (Nov 2017)**

There are various software development approaches defined and designed which are used/employed during development process of software, these approaches are also referred as “Software Development Process Models” (e.g. **Waterfall model, incremental model, V-model, iterative model, RAD model, Agile model, Spiral model, Prototype model** etc.). Each process model follows a particular life cycle in order to ensure success in process of software development.

Software life cycle models describe phases of the software cycle and the order in which those phases are executed. Each phase produces deliverables required by the next phase in the life cycle. Requirements are translated into design. Code is produced according to the design which is called development phase. After coding and development the testing verifies the deliverable of the implementation phase against requirements. The testing team follows Software Testing Life Cycle (STLC) which is similar to the development cycle followed by the development team.

There are following six phases in every Software development life cycle model:

- Requirement gathering and analysis
- Design
- Implementation or coding
- Testing
- Deployment
- Maintenance

**1) Requirement gathering and analysis:** Business requirements are gathered in this phase. This phase is the main focus of the project managers and stake holders. Meetings with managers, stake holders and users are held in order to determine the requirements like; Who is going to use the system? How will they use the system? What data should be input into the system? What data should be output by the system? These are general questions that get answered during a requirements gathering phase. After requirement gathering these requirements are analyzed for their validity and the possibility of incorporating the requirements in the system to be development is also studied.

Finally, a Requirement Specification document is created which serves the purpose of guideline for the next phase of the model. The testing team follows the Software Testing Life Cycle and starts the Test Planning phase after the requirements analysis is completed.

**2) Design:** In this phase the system and software design is prepared from the requirement specifications which were studied in the first phase. System Design helps in specifying hardware and system requirements and also helps in defining overall system architecture. The system design specifications serve as input for the next phase of the model.

In this phase the testers comes up with the Test strategy, where they mention what to test, how to test.



**3) Implementation / Coding:** On receiving system design documents, the work is divided in modules/units and actual coding is started. Since, in this phase the code is produced so it is the main focus for the developer. This is the longest phase of the software development life cycle.

**4) Testing:** After the code is developed it is tested against the requirements to make sure that the product is actually solving the needs addressed and gathered during the requirements phase. During this phase all types of functional testing like unit testing, integration testing, system testing, acceptance testing are done as well as non-functional testing are also done.

**5) Deployment:** After successful testing the product is delivered / deployed to the customer for their use.

As soon as the product is given to the customers they will first do the beta testing. If any changes are required or if any bugs are caught, then they will report it to the engineering team. Once those changes are made or the bugs are fixed then the final deployment will happen.

**6) Maintenance:** Once when the customers starts using the developed system then the actual problems comes up and needs to be solved from time to time. This process where the care is taken for the developed product is known as maintenance.